

GDS Import/Export User Manual

Layout Automation Toolkit

Version: 1.0 **Date:** January 5, 2026 **Author:** Layout Automation Team

Table of Contents

1. [Overview](#)
 2. [Getting Started](#)
 3. [GDS Export](#)
 4. [GDS Import](#)
 5. [Fixed Issues](#)
 6. [Best Practices](#)
 7. [Troubleshooting](#)
 8. [API Reference](#)
-

Overview

The Layout Automation Toolkit provides robust GDS-II file format support for exporting and importing integrated circuit layouts. GDS-II (Graphic Data System) is the industry-standard format for IC mask layout data exchange.

Key Features

- **Hierarchical cell structure support** - Preserve parent-child relationships

- **Accurate positioning** - Zero-shift import/export with relative coordinates
- **Cell reuse optimization** - Efficient handling of repeated cell instances
- **Multi-layer support** - Full layer mapping with FreePDK45 technology
- **Collision handling** - Automatic unique naming for cells with identical names

Technology Requirements

- Python 3.x
- gdstk library (GDS manipulation)
- FreePDK45 technology file (for layer definitions)

Getting Started

Installation

```
# Install the layout automation toolkit
pip install -e .

# Verify installation
python -c "from layout_automation import Cell; print('Installation successful!')"
```

Basic Usage

```
from layout_automation import Cell

# Create a simple cell
cell = Cell('my_design', 'metal1')
cell.pos_list = [0, 0, 100, 100]

# Export to GDS
cell.to_gds('output.gds')

# Import from GDS
imported_cell = Cell.from_gds('output.gds')
```

GDS Export

Simple Export

Export a single cell or hierarchical structure to GDS format:

```
# Create cells
top = Cell('top')
block1 = Cell('block1', 'metal1')
block2 = Cell('block2', 'metal2')

# Build hierarchy
top.add_instance([block1, block2])

# Set positions
top.pos_list = [0, 0, 1000, 1000]
block1.pos_list = [100, 100, 300, 300]
block2.pos_list = [400, 400, 600, 600]

# Export
top.to_gds('design.gds')
```

Export with Custom Layer Mapping

```
# Define custom layer mapping
custom_layers = {
    'metal1': (10, 0),
    'metal2': (20, 0),
    'via1': (30, 0)
}

# Export with custom mapping
cell.to_gds('output.gds', layer_map=custom_layers, use_tech_file=False)
```

Hierarchical Export

The toolkit automatically handles hierarchical structures:

```
# 3-level hierarchy example
top = Cell('chip')
mid1 = Cell('block_a')
mid2 = Cell('block_b')
leaf1 = Cell('transistor', 'diffusion')
leaf2 = Cell('contact', 'metal1')

# Build hierarchy
mid1.add_instance(leaf1)
mid2.add_instance(leaf2)
top.add_instance([mid1, mid2])

# Set positions (all positions are absolute in your design)
top.pos_list = [0, 0, 5000, 5000]
mid1.pos_list = [500, 500, 2000, 2000]
mid2.pos_list = [3000, 3000, 4500, 4500]
leaf1.pos_list = [600, 600, 800, 800]
leaf2.pos_list = [3100, 3100, 3300, 3300]

# Export - positions are automatically converted to relative coordinates
top.to_gds('chip.gds')
```

Key Point: You always work with absolute coordinates in your Python code. The GDS export automatically converts them to relative coordinates for proper hierarchical representation.

GDS Import

Basic Import

```
# Import top cell from GDS file
cell = Cell.from_gds('design.gds')

# Import specific cell by name
cell = Cell.from_gds('design.gds', cell_name='my_block')
```

Import with Layer Mapping

```
# Define reverse layer mapping (GDS layer -> layer name)
layer_map = {
    (10, 0): 'metal1',
    (20, 0): 'metal2',
    (30, 0): 'via1'
}

# Import with custom mapping
cell = Cell.from_gds('design.gds', layer_map=layer_map, use_tech_file=False)
```

Accessing Imported Data

```
# Import cell
top = Cell.from_gds('chip.gds')

# Access cell properties
print(f"Cell name: {top.name}")
print(f"Position: {top.pos_list}")
print(f"Number of children: {len(top.children)}")

# Traverse hierarchy
for child in top.children:
    print(f"  Child: {child.name}, Position: {child.pos_list}")
    if child.is_leaf:
        print(f"    Leaf on layer: {child.layer_name}")
```

Fixed Issues

This section documents critical issues that have been identified and fixed in the GDS import/export implementation.

Issue 1: Hierarchical Position Shift (FIXED ✓)

Problem: When exporting hierarchical designs to GDS and re-importing them, child cells would appear at incorrect positions, shifted from their original locations.

Root Cause: The GDS export was using absolute coordinates for cell references instead of relative coordinates. For example, if a parent cell was at (100, 100) and contained a child at (150, 150), the GDS reference was incorrectly using (150, 150) instead of the relative offset (50, 50).

Solution: Modified `_convert_to_gds()` method in `layout_automation/cell.py:1806-1855` to calculate relative positions:

```
# Get parent's origin
parent_x1 = self.pos_list[0] if all(v is not None for v in self.pos_list) else 0
parent_y1 = self.pos_list[1] if all(v is not None for v in self.pos_list) else 0

# Create reference using RELATIVE position
x1, y1, _, _ = child.pos_list
ref = gdstk.Reference(child_gds_cell, origin=(x1 - parent_x1, y1 - parent_y1))
```

Impact: Zero position shift in all hierarchical exports. All cells maintain exact positions after export/import cycle.

Test Coverage: `test_gds_hierarchy_absolute.py`

Issue 2: Cell Name Collision Causing Data Loss (FIXED)

✓)

Problem: When different cells had the same name (e.g., two different "rect" cells with different sizes in different blocks), only one would survive in the GDS file. The second cell would overwrite the first, causing data loss.

Root Cause: The implementation used `cell.name` as the dictionary key in `gds_cells_dict`, so cells with identical names would overwrite each other:

```
# OLD (BROKEN) CODE:
gds_cells_dict[cell.name] = gds_cell # Second 'rect' overwrites first!
```

Solution: Changed to use Python object ID as the dictionary key, and implemented automatic unique name generation:

```
# Use object ID as key (layout_automation/cell.py:1786-1804)
cell_id = id(self)

# Skip if already converted (enables cell reuse)
if cell_id in gds_cells_dict:
    return gds_cells_dict[cell_id]

# Generate unique GDS name when collision occurs
gds_cell_name = self.name
if gds_cell_name in gds_name_counter:
    gds_name_counter[gds_cell_name] += 1
    gds_cell_name = f"{self.name}_{gds_name_counter[gds_cell_name]}"
else:
    gds_name_counter[gds_cell_name] = 0

gds_cells_dict[cell_id] = gds_cell # Key by ID, not name!
```

Impact: - All cells with unique content are preserved - Cells get unique GDS names: `rect`, `rect_1`, `rect_2`, etc. - Proper cell reuse: same object used multiple times → single GDS cell

Test Coverage: `test_gds_name_collision.py`, `test_gds_cell_reuse.py`

Issue 3: Cell Name 'top' (NO ISSUE FOUND ✓)

Initial Report: User reported that cell name 'top' might not work correctly.

Investigation: Comprehensive testing showed no issues with the name 'top'. It works correctly in all scenarios.

Conclusion: No fix required. The name 'top' is safe to use.

Test Coverage: `test_gds_top_cell.py`

Best Practices

Cell Naming

```
# ✓ GOOD: Descriptive, unique names when cells have different content
block_a = Cell('memory_block', 'metal1')
block_b = Cell('logic_block', 'metal2')

# ✓ GOOD: Same name OK if you intend to reuse the same cell
shared = Cell('standard_cell', 'metal1')
block1.add_instance(shared) # Reuse same object
block2.add_instance(shared) # Will share one GDS cell

# △ ACCEPTABLE: Same name for different cells
# (Toolkit auto-generates unique names: rect, rect_1)
rect_a = Cell('rect', 'metal1') # 100x100
rect_b = Cell('rect', 'metal2') # 200x200 - different content
```

Position Management

```
# ✓ GOOD: Always use absolute coordinates in Python
top.pos_list = [0, 0, 1000, 1000]
child.pos_list = [100, 100, 300, 300] # Absolute, not relative to parent

# The toolkit handles conversion to relative coordinates automatically

# ✓ GOOD: Ensure all position values are set before export
if all(v is not None for v in cell.pos_list):
    cell.to_gds('output.gds')
```

Hierarchical Design

```
# ✓ GOOD: Build hierarchy with add_instance()
parent.add_instance(child)
parent.add_instance([child1, child2, child3])

# ✓ GOOD: Set positions after building hierarchy
parent.pos_list = [0, 0, 1000, 1000]
child.pos_list = [100, 100, 500, 500]

# Export preserves entire hierarchy
parent.to_gds('design.gds')
```

Layer Mapping

```
# ✓ GOOD: Use technology file for standard designs
cell.to_gds('output.gds', use_tech_file=True) # Default

# ✓ GOOD: Custom mapping for special cases
custom = {
    'layer1': (10, 0),
    'layer2': (20, 0)
}
cell.to_gds('output.gds', layer_map=custom, use_tech_file=False)
```

Troubleshooting

Position Mismatch After Import

Symptom: Cells appear at wrong positions after export/import cycle.

Diagnosis:

```
# Export and re-import
original.to_gds('test.gds')
imported = Cell.from_gds('test.gds')

# Check positions
print(f"Original: {original.pos_list}")
print(f"Imported: {imported.pos_list}")
```

Solution: This should not occur with current implementation. If you see this: 1. Verify you're using the latest version with hierarchical fix 2. Check that all cells have valid `pos_list` values 3. Report as a bug with test case

Missing Cells After Export

Symptom: Some cells don't appear in the GDS file.

Common Causes: 1. **Unset positions:** Cells with `None` values in `pos_list` are skipped 2. **Not in hierarchy:** Cell exists but isn't added to any parent

Diagnosis:

```
# Check all cells have positions
for child in cell.children:
    if any(v is None for v in child.pos_list):
        print(f"Warning: {child.name} has incomplete position")
```

Solution:

```
# Ensure all positions are set
child.pos_list = [x1, y1, x2, y2] # All values must be numbers

# Ensure cells are in hierarchy
parent.add_instance(child)
```

Cell Name Conflicts

Symptom: Warning about duplicate cell names or unexpected name changes in GDS.

Explanation: Different cells with the same name are automatically given unique GDS names to prevent data loss.

Example:

```
# Two different cells, same name
rect1 = Cell('rect', 'metal1') # 100x100
rect2 = Cell('rect', 'metal2') # 200x200

block1.add_instance(rect1)
block2.add_instance(rect2)
top.add_instance([block1, block2])
top.to_gds('output.gds')

# GDS file contains:
# - 'rect' (first occurrence)
# - 'rect_1' (second occurrence)
```

This is expected behavior and ensures all your data is preserved.

Layer Not Appearing

Symptom: Exported GDS is missing certain layers.

Diagnosis:

```
# Check if layer is defined
from layout_automation import tech
print(tech.layers) # Shows all available layers
```

Solutions: 1. Use a layer name that exists in FreePDK45 technology 2. Provide custom layer mapping 3. Check layer name spelling (case-sensitive)

API Reference

Cell.to_gds()

Export cell and its hierarchy to GDS-II format.

```
def to_gds(self, filename: str,
            layer_map: Optional[Dict[str, Tuple[int, int]]] = None,
            use_tech_file: bool = True) -> None
```

Parameters: - `filename` (str): Output GDS file path - `layer_map` (dict, optional): Custom layer mapping {layername: (layernum, datatype)} - `use_tech_file` (bool): Use FreePDK45 technology file for layer mapping (default: True)

Example:

```
cell.to_gds('output.gds')
cell.to_gds('output.gds', layer_map={'metall1': (10, 0)}, use_tech_file=False)
```

Cell.from_gds()

Import cell from GDS-II file format.

```
@classmethod
def from_gds(cls, filename: str,
             cell_name: Optional[str] = None,
             layer_map: Optional[Dict[Tuple[int, int], str]] = None,
             use_tech_file: bool = True) -> 'Cell'
```

Parameters: - `filename` (str): Input GDS file path - `cell_name` (str, optional): Specific cell to import (default: top cell) - `layer_map` (dict, optional): Custom layer mapping {(layer, datatype): layer_name} - `use_tech_file` (bool): Use FreePDK45 technology file (default: True)

Returns: - `Cell`: Imported cell with full hierarchy

Example:

```
cell = Cell.from_gds('input.gds')
cell = Cell.from_gds('input.gds', cell_name='my_block')
reverse_map = {(10, 0): 'metal1', (20, 0): 'metal2'}
cell = Cell.from_gds('input.gds', layer_map=reverse_map, use_tech_file=False)
```

Testing

The toolkit includes comprehensive test coverage for GDS functionality:

Test Files

1. [testgdshierarchy_absolute.py](#) - Validates hierarchical position preservation
2. [testgdsname_collision.py](#) - Tests cell name collision handling
3. [testgdscell_reuse.py](#) - Verifies proper cell reuse vs collision scenarios
4. [testgdstop_cell.py](#) - Confirms 'top' cell name works correctly

Running Tests

```
# Run individual test
python test_gds_hierarchy_absolute.py

# Run all GDS tests
python -m pytest test_gds_*.py -v

# Run with coverage
python -m pytest test_gds_*.py --cov=layout_automation --cov-report=html
```

Changelog

Version 1.0 (January 2026)

Major Fixes: - Fixed hierarchical position shift by implementing relative coordinate conversion - Fixed cell name collision data loss by using object IDs as dictionary keys - Added automatic unique name generation for conflicting cell names - Improved cell reuse detection for optimal GDS structure

Testing: - Added comprehensive test suite for all GDS operations - Validated zero-shift hierarchical export/import - Verified cell name collision handling - Confirmed proper cell reuse behavior

Documentation: - Created comprehensive user manual - Added GDS investigation summary - Documented all fixed issues and solutions

Support

For issues, questions, or contributions:

- **GitHub Repository:** https://github.com/StevenJWChen/layout_automation
 - **Issue Tracker:** https://github.com/StevenJWChen/layout_automation/issues
 - **Branch:** `claude/create-user-manual-011CULDvXyEdpVuUMmBHfwL`
-

License

This toolkit is part of the Layout Automation project.

End of User Manual